UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Rizky Apryandi Triadmojo (2428240107)
Kelas	Sistem Informasi
Judul project / topik API	Klinik - RESTful API data dokter dengan filter berdasarkan spesialis
Tautan repositori	https://github.com/rizkyapryndii/tugas1-restful-2428240107-rizkyapryanditriadmojo
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	19 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

API Klinik mengelola data dokter dengan atribut id, nama, spesialis, nomor STR, jadwal praktik, dan biaya konsultasi. Pengujian berfokus pada operasi CRUD serta filter daftar dokter berdasarkan query spesialis. Collection menggunakan base URL http://localhost:3000; operasi penulisan memakai header x-api-key.

1.2 Daftar endpoint

API key diwajibkan untuk POST, PUT, dan DELETE; GET daftar, detail, dan filter tidak memerlukan API key berdasarkan bukti request.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/doctors	Tidak	200	Mengambil seluruh data dokter
2	GET	/doctors/:id	Tidak	200	Mengambil detail dokter berdasarkan ID
3	GET	/doctors?spesialis=:spesialis	Tidak	200	Memfilter dokter berdasarkan spesialis
4	POST	/doctors	Ya	201	Menambahkan data dokter
5	PUT	/doctors/:id	Ya	200	Memperbarui data dokter
6	DELETE	/doctors/:id	Ya	204	Menghapus data dokter
7	POST	/doctors (tanpa API key)	Ya	401	Memastikan penulisan ditolak tanpa API key
8	POST	/doctors (body tidak valid)	Ya	400	Memeriksa validasi body permintaan
9	GET	/doctors/999	Tidak	404	Memastikan ID yang tidak ada menghasilkan 404
10	DELETE	/doctors/:id (ID tidak ada)	Ya	404	Memeriksa penghapusan ID yang tidak ditemukan

1.3 Variabel Environment

Environment yang terlihat bernama Doctor API Environment. Variabel yang digunakan: base_url = http://localhost:3000 dan api_key = disimpan sebagai nilai rahasia (tidak dicantumkan). Variabel doctorId digunakan pada URL request PUT/DELETE bila diatur di Environment.

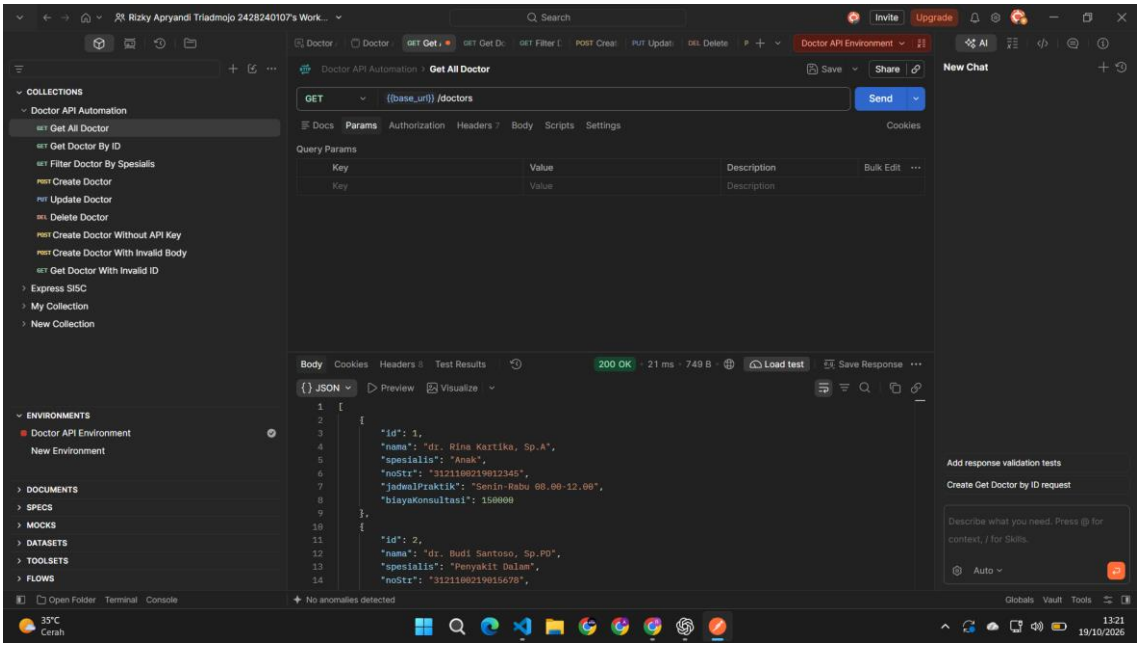
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat API lokal
2	api_key	(disimpan rahasia)	Header autentikasi x-api-key
3	doctorId	(diambil dari respons)	ID dokter untuk request lanjutan
4	—	—	Tidak ada variabel lain yang terlihat

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Screenshot pertama memperlihatkan Collection Doctor API Automation dengan sepuluh request. Permintaan tersusun dari operasi utama dan skenario negatif.



Gambar 1. Collection beserta seluruh request

Request dikelompokkan dalam Collection Doctor API Automation. Enam request pertama menguji operasi utama, sedangkan request berikutnya mencakup kasus tanpa API key, body tidak valid, ID dokter tidak valid, dan ID hapus yang tidak ditemukan.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	Get All Doctor	GET	{{base_url}}/doctors	Doctor API Automation
2	Get Doctor By ID	GET	{{base_url}}/doctors/1	Doctor API Automation

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
3	Filter Doctor By Spesialis	GET	{{base_url}}/doctors?spesialis=anak	Doctor API Automation
4	Create Doctor	POST	{{base_url}}/doctors	Doctor API Automation
5	Update Doctor	PUT	{{base_url}}/doctors/{{doctorId}}	Doctor API Automation
6	Delete Doctor	DELETE	{{base_url}}/doctors/{{doctorId}}	Doctor API Automation
7	Create Doctor Without API Key	POST	{{base_url}}/doctors	Doctor API Automation
8	Create Doctor With Invalid Body	POST	{{base_url}}/doctors	Doctor API Automation
9	Get Doctor With Invalid ID	GET	{{base_url}}/doctors/999	Doctor API Automation
10	Delete Doctor With Invalid ID	DELETE	{{base_url}}/doctors/{{doctorId}}	Doctor API Automation

Bab 3 Automated Test Script

Setiap request sebaiknya memiliki assertion status HTTP dan pemeriksaan bentuk/isi JSON yang sesuai dengan respons endpoint. Hasil Runner pada bukti terakhir masih menunjukkan kegagalan assertion sehingga pengujian belum seluruhnya lulus.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	Get All Doctor	200; status sukses	Body berupa array; setiap item memiliki id dan spesialis
2	Get Doctor By ID	200	Body objek dokter memiliki id dan nama
3	Filter Doctor By Spesialis	200	Body array; setiap spesialis sesuai query anak
4	Create Doctor	201	status success; data berisi id dan nama
5	Update Doctor	200	status success; data.id sesuai ID yang diubah
6	Delete Doctor	204	Respons tanpa body
7	Create Doctor Without API Key	401	status error; pesan API key wajib disertakan
8	Create Doctor With Invalid Body	400 (target)	Respons validasi error
9	Get Doctor With Invalid ID	404	status error; pesan data tidak ditemukan
10	Delete Doctor With Invalid ID	404	status error; pesan data tidak ditemukan

3.2 Cuplikan 1: request GET

Skrip GET detail di bawah memeriksa status 200 dan properti objek dokter. Respons /doctors/:id berupa satu objek, sedangkan GET daftar dan filter mengembalikan array.

```
pm.test('Status code harus 200', function () {
  pm.response.to.have.status(200);
});

const dokter = pm.response.json();
pm.test('Respons detail berupa objek dokter', function () {
  pm.expect(dokter).to.be.an('object');
  pm.expect(dokter).to.have.property('id');
  pm.expect(dokter).to.have.property('spesialis');
});
```

Assertion pertama memastikan endpoint detail merespons HTTP 200. Assertion kedua memeriksa bahwa body berupa objek dengan properti id dan spesialis; endpoint detail mengembalikan satu objek, bukan array.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Skrip POST di bawah memeriksa status 201, pesan sukses, dan data dokter yang dibuat. ID respons disimpan sebagai doctorId agar dapat digunakan request berikutnya.

```
pm.test('Status code harus 201 Created', function () {
  pm.response.to.have.status(201);
});

const hasil = pm.response.json();
pm.test('Data dokter berhasil dibuat', function () {
  pm.expect(hasil.status).to.eql('success');
  pm.expect(hasil.data).to.have.property('id');
  pm.expect(hasil.data).to.have.property('nama');
});

if (pm.response.code === 201 && hasil.data && hasil.data.id) {
  pm.environment.set('doctorId', hasil.data.id);
}
```

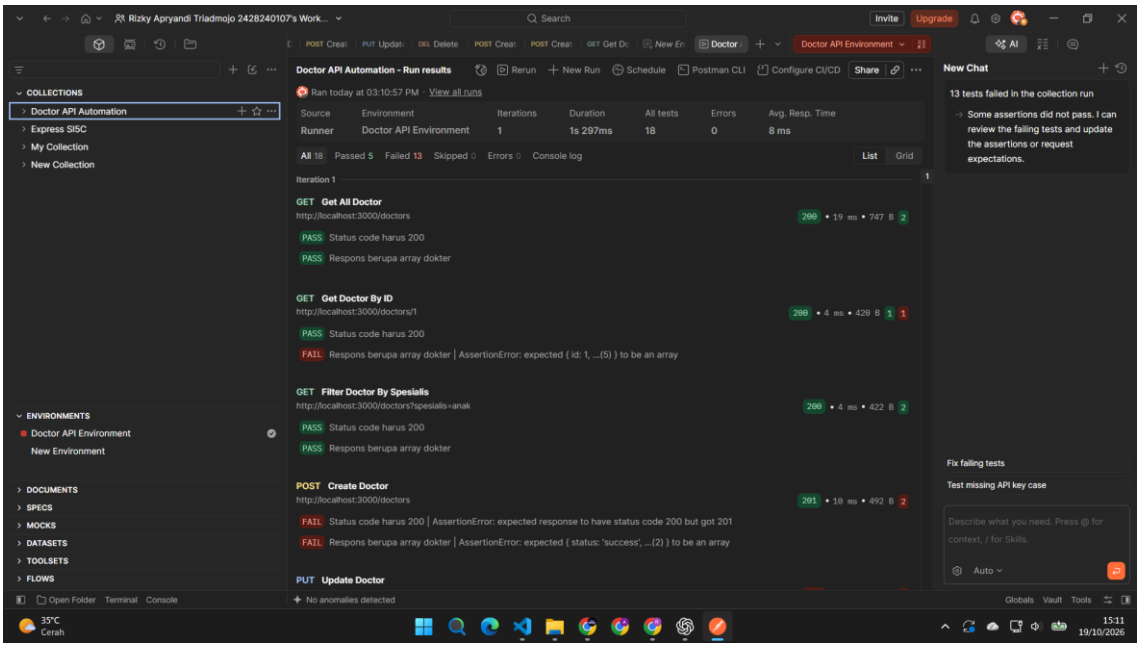
Skrip menyimpan id dari hasil.data sebagai doctorId hanya jika respons 201 dan ID tersedia. Request Update Doctor dan Delete Doctor memakai {{doctorId}} pada URL. Jalankan POST lebih dahulu; bila urutannya ditukar, variabel mungkin kosong atau masih berisi ID lama.

Bab 4 Hasil Collection Runner

Ringkasan Collection Runner yang tersedia menunjukkan 18 test: 5 lulus dan 13 gagal. Karena belum 0 gagal, hasil ini belum memenuhi syarat kelulusan Runner pada template.

4.1 Ringkasan Collection Runner

Tangkapan layar menunjukkan Collection Doctor API Automation, Environment Doctor API Environment, 18 test, 5 lulus, dan 13 gagal. Jalankan ulang setelah menyesuaikan assertion agar sesuai dengan respons API.



Gambar 2. Ringkasan hasil Collection Runner

Runner menjalankan 10 request dalam satu iterasi dan menghitung 18 test. Hasil yang tampak adalah 5 lulus dan 13 gagal. Bukti ini belum menunjukkan seluruh test lulus; assertion perlu diperbaiki lalu Collection dijalankan ulang.

4.2 Rekap angka hasil

Nama Collection	Doctor API Automation
Environment yang dipakai	Doctor API Environment
Jumlah request dijalankan	10 request (18 test assertion)
Jumlah test lulus	5
Jumlah test gagal	13

Tanggal dan jam pengujian	19 Oktober 2026, sekitar 15:10 WIB (tangkapan layar)

4.3 Temuan selama pengujian

Kegagalan yang tampak mencakup assertion GET detail yang mengharapkan array padahal API mengembalikan objek, serta assertion POST yang mengharapkan status 200 dan array padahal respons sukses adalah 201 Created dengan objek data. Respons DELETE untuk ID yang tidak ditemukan menghasilkan 404; request tanpa API key menghasilkan 401. Pesan respons terlihat, tetapi seluruh 13 kegagalan tidak dapat diidentifikasi satu per satu dari tangkapan layar Runner yang diberikan.

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Get Doctor By ID	Respons berupa array dokter; aktualnya objek	Assertion bentuk respons tidak sesuai untuk endpoint detail	Ubah assertion agar memeriksa objek dokter dan properti yang tersedia.
2	Create Doctor	Status diharapkan 200, aktual 201; respons array diharapkan, aktual objek berisi status dan data	Assertion masih mengikuti pola status/list yang keliru	Harapkan 201 Created dan periksa status success serta properti data.

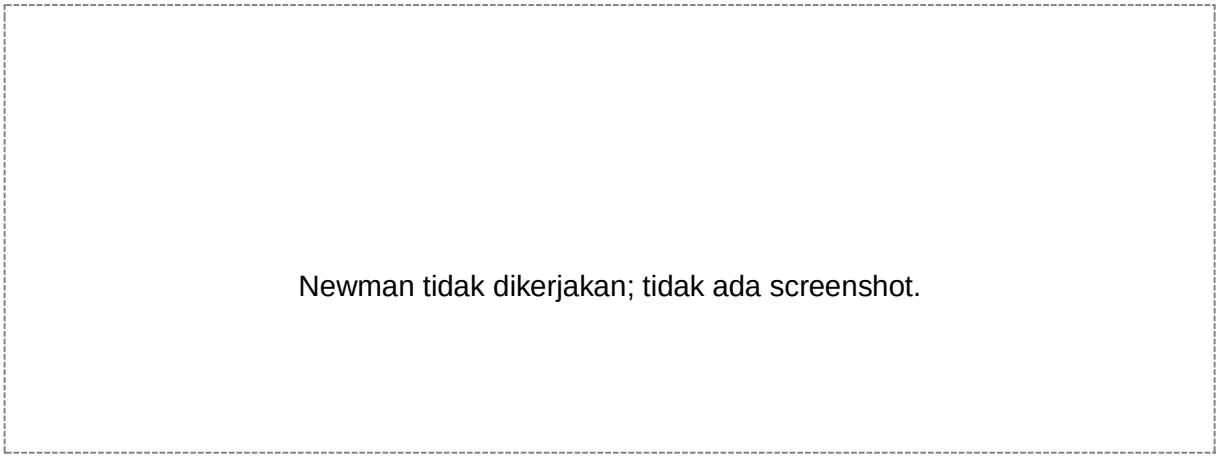
Bab 5 Newman di Terminal (Pengayaan)

> Newman tidak dikerjakan pada bukti yang diberikan.

Newman tidak dikerjakan; tidak ada tangkapan layar hasil Newman dalam bukti yang diberikan.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.



Gambar 3. Hasil menjalankan Collection dengan Newman

Tidak ada perintah Newman atau exit code yang dapat dilaporkan karena pengujian Newman tidak disertakan.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/rizkyapryndii/tugas1-restful-2428240107-rizkyapryanditriadmojo
Lokasi berkas Collection (.json)	Belum dicantumkan; ekspor Collection sebagai JSON dari Postman.
Lokasi berkas Environment (.json)	Belum dicantumkan; ekspor Environment tanpa nilai api_key asli.
Berkas Environment tidak memuat kunci rahasia sungguhan	[✓] Ya [ ] Tidak (pastikan sebelum dibagikan)
Repositori dapat dibuka dosen	[ ] Ya [ ] Tidak (belum diverifikasi)

Repositori proyek: https://github.com/rizkyapryndii/tugas1-restful-2428240107-rizkyapryanditriadmojo. Collection dan Environment dapat disimpan sebagai berkas JSON dari Postman. Nilai API key harus disamarkan sebelum berkas Environment dibagikan.

Bab 7 Refleksi

Pengujian dilakukan pada API klinik yang mengelola data dokter. Endpoint yang diuji mencakup daftar, detail, filter spesialis, tambah, ubah, dan hapus dokter. Bukti yang tersedia berasal dari Postman dan menunjukkan bahwa sebagian assertion masih perlu disesuaikan dengan bentuk respons aktual.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Assertion pemeriksaan bentuk respons paling membantu karena menemukan perbedaan antara endpoint daftar yang mengembalikan array dan endpoint detail yang mengembalikan satu objek. Dengan begitu, kesalahan assertion dapat dibedakan dari kesalahan pada API.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Pengujian manual membantu melihat respons satu per satu, sedangkan Collection Runner menjalankan banyak request secara berurutan dan merangkum hasilnya. Bagian yang paling perlu diperhatikan adalah menyamakan status dan struktur respons yang diharapkan pada skrip dengan hasil nyata dari API.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Ya
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Tidak - perlu diperbaiki
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ya
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Belum terverifikasi
7	Server dinyalakan ulang sebelum Runner dijalankan	Tidak terverifikasi
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Tidak - 13 gagal
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Belum terverifikasi
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Belum dilakukan

Lampiran B Kriteria Penilaian

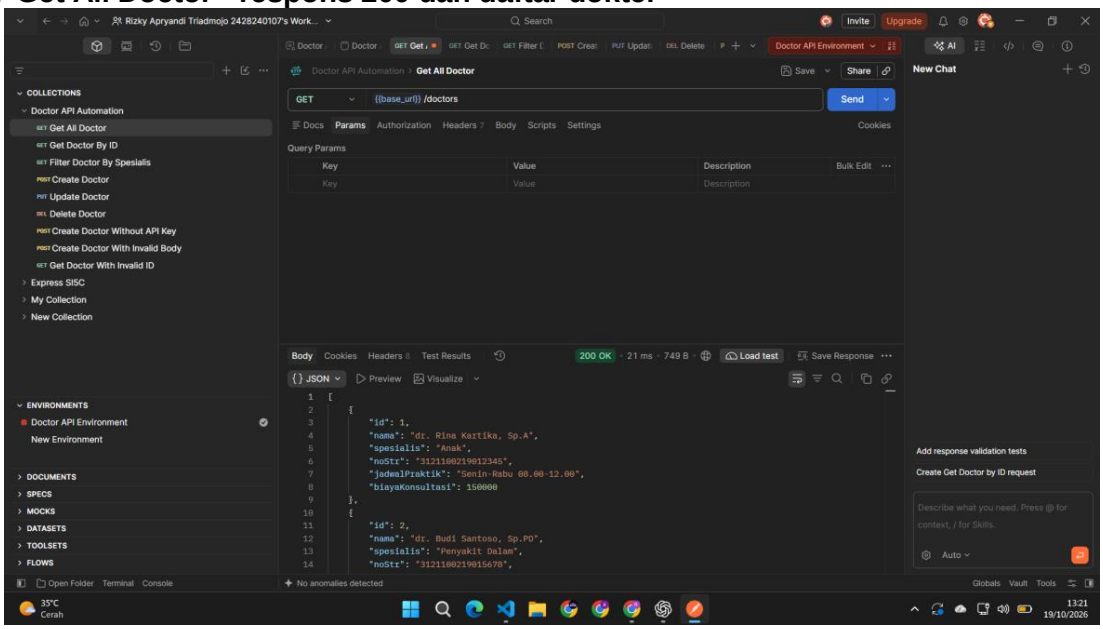
Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen

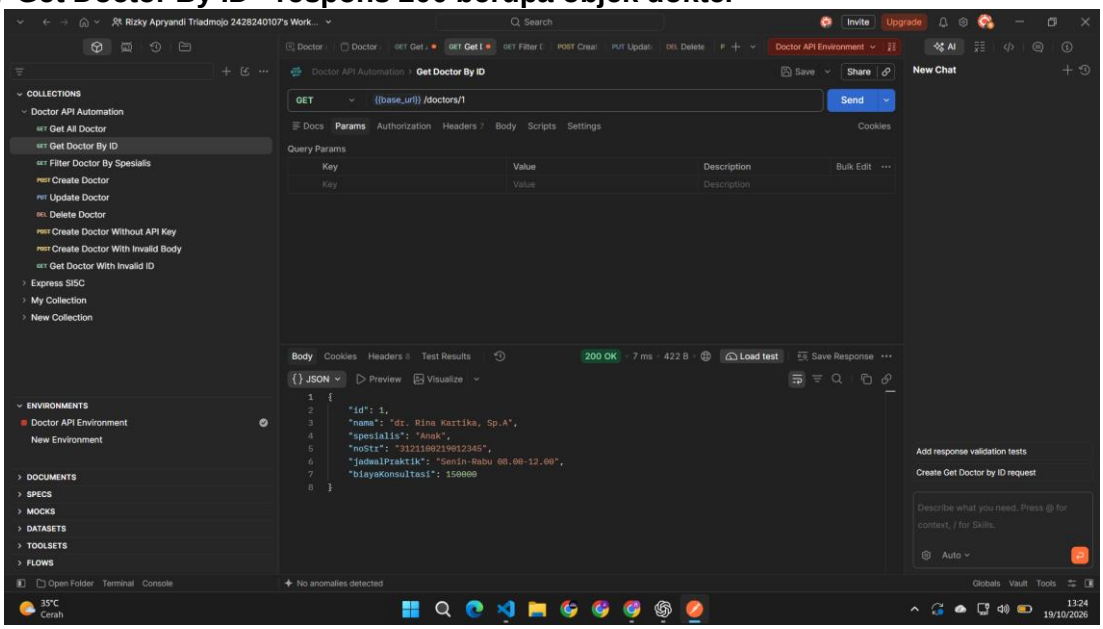
Lampiran C Tangkapan Layar Pengujian Postman

Bukti berikut memperlihatkan request, respons, dan ringkasan Collection Runner yang digunakan dalam laporan.

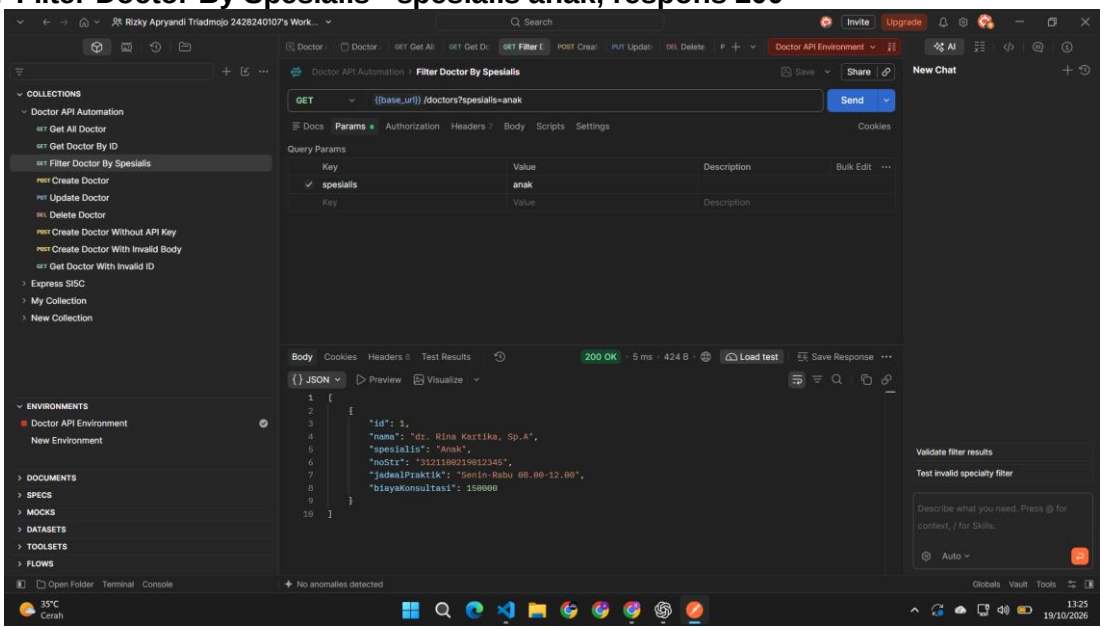
GET Get All Doctor - respons 200 dan daftar dokter



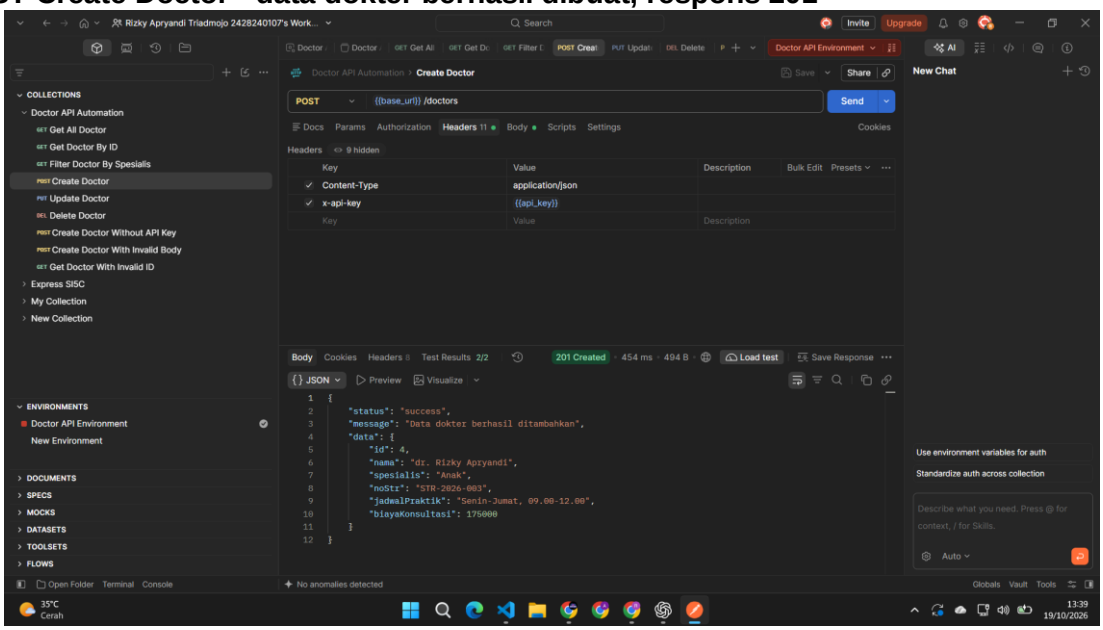
GET Get Doctor By ID - respons 200 berupa objek dokter



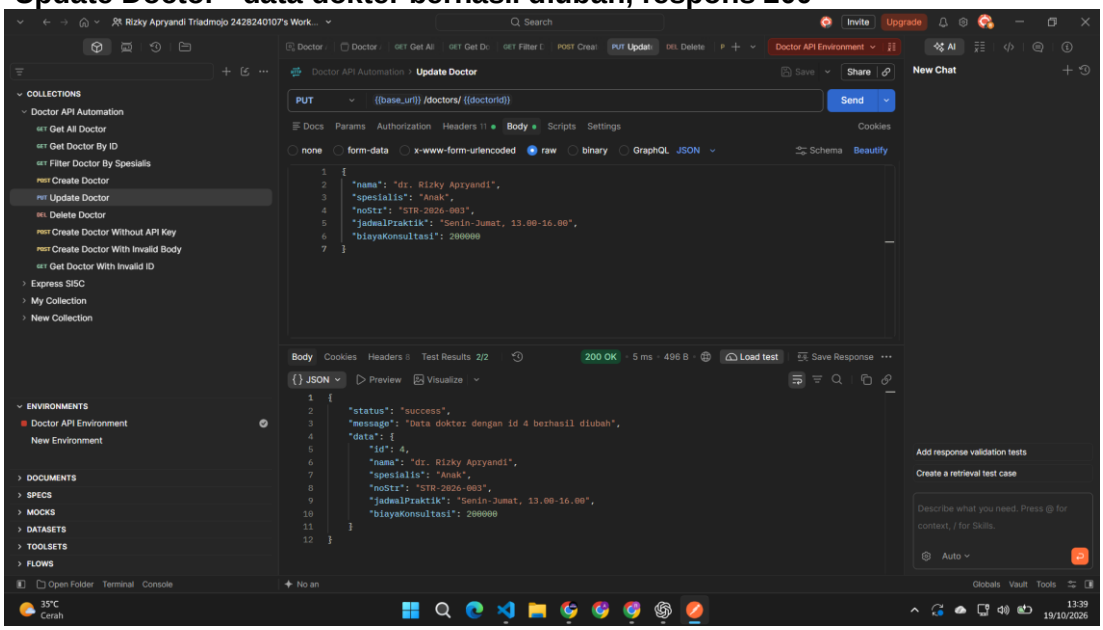
GET Filter Doctor By Spesialis - spesialis anak, respons 200



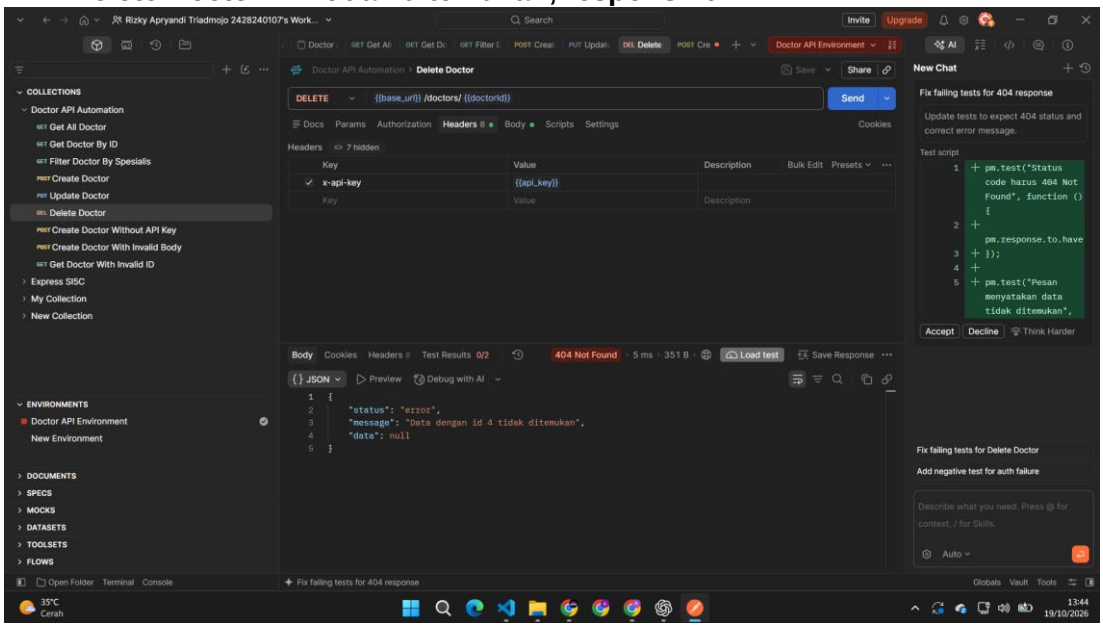
POST Create Doctor - data dokter berhasil dibuat, respons 201



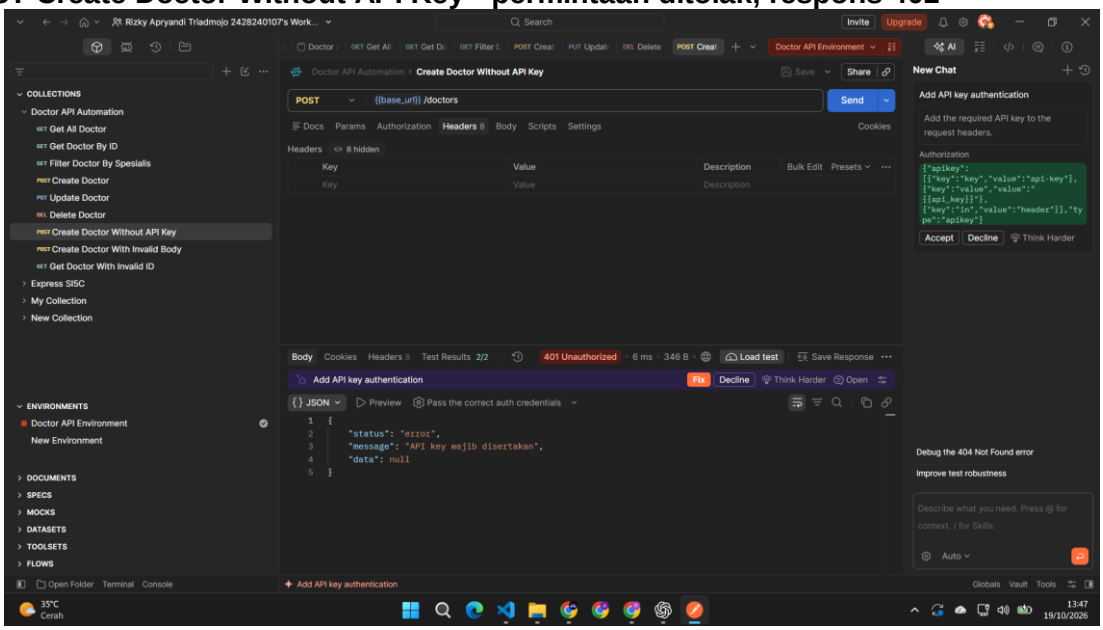
PUT Update Doctor - data dokter berhasil diubah, respons 200



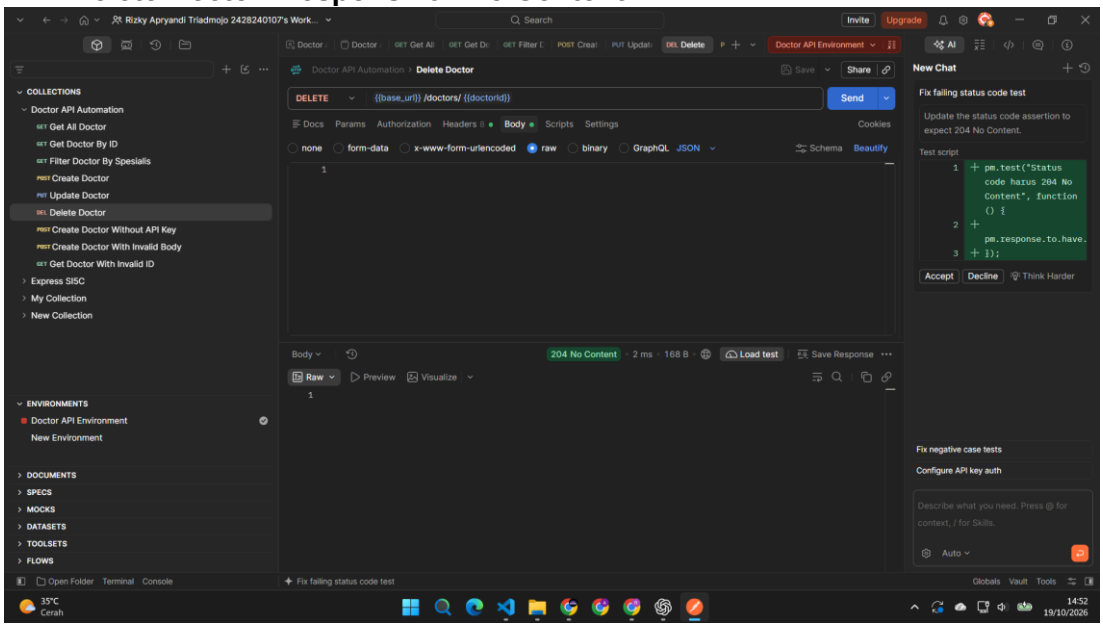
DELETE Delete Doctor - ID tidak ditemukan, respons 404



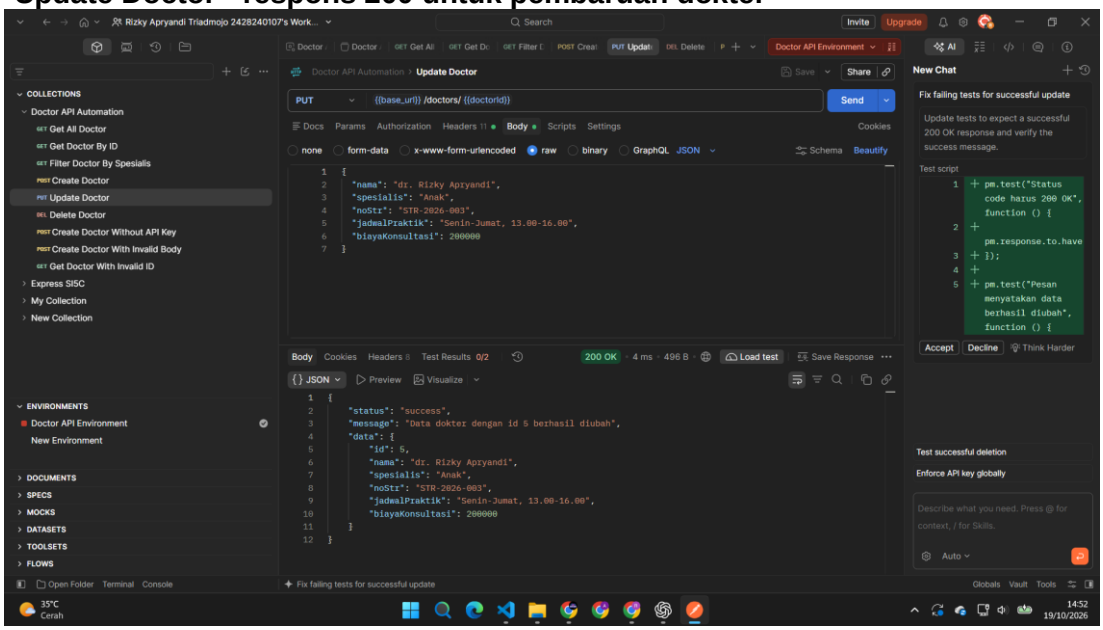
POST Create Doctor Without API Key - permintaan ditolak, respons 401



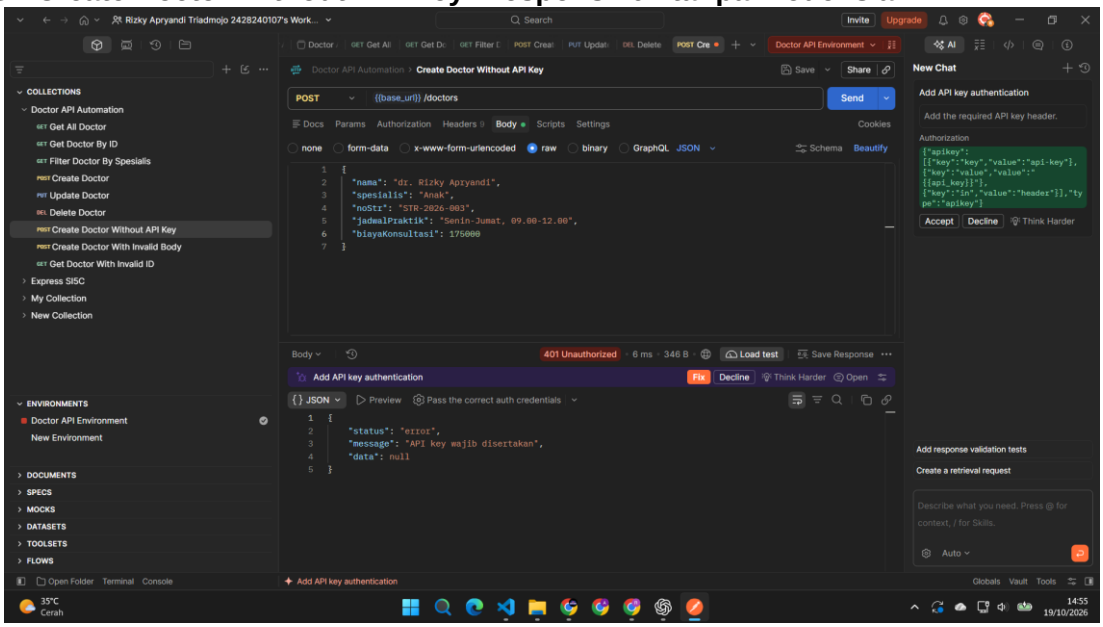
DELETE Delete Doctor - respons 204 No Content



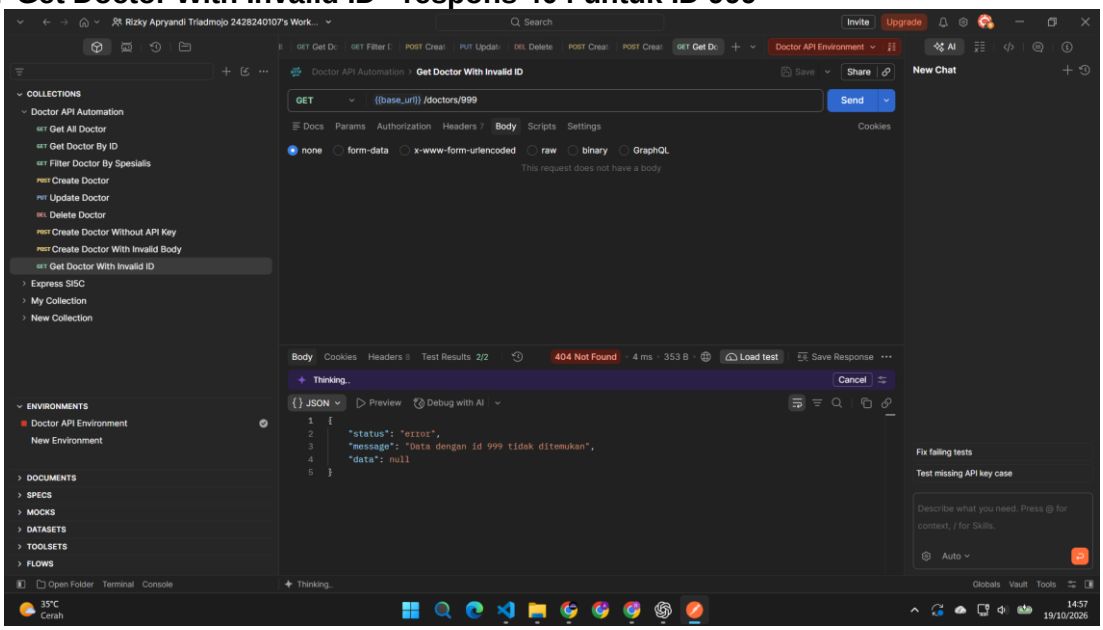
PUT Update Doctor - respons 200 untuk pembaruan dokter



POST Create Doctor Without API Key - respons 401 tanpa kredensial



GET Get Doctor With Invalid ID - respons 404 untuk ID 999



Collection Runner - 18 test, 5 lulus, 13 gagal

